FIREWALL POLICY COMPILER

AUTOMATED NETWORK SECURITY SYSTEM

System Overview & Technical Documentation

Version	1.0
Date	May 2026
Supervisor	Dr. Mohamed El-Mowafy Dr. Mohamed El-Nahas Dr. Mohamed Noureldin
Classification	Confidential — *INSIDER-EYE TEAM*
Purpose	Educational, Security Research & Network Threat Mitigation
Platform	Linux (Docker) | Python 3.10+ | Flask | iptables / nftables
Enforcement	Real-time kernel-level packet filtering via docker exec injection
Contribution	Ahmed Eldesouky Mohamed Salah Ahmed Elasmar Hazem Ashraf Ahmed Abdelmoniem

1. Executive Summary

Firewall Policy Compiler is a full-stack, Docker-based security engineering platform that closes the gap between human-readable firewall policy definitions and real, kernel-enforced Linux network rules. Rather than simulating firewall behaviour, it compiles user-defined YAML policies and injects the resulting iptables/nftables commands directly into a running containerised network — with verification via live connectivity testing and attack simulation.

The system's core philosophy is a "dual-layer enforcement model": abstract semantic policies authored in YAML are compiled by the backend engine and applied to a physically isolated multi-container environment. Every rule is validated in real time through genuine network tests — not approximations.

Figure 1.1 — Main Policy Engine Dashboard

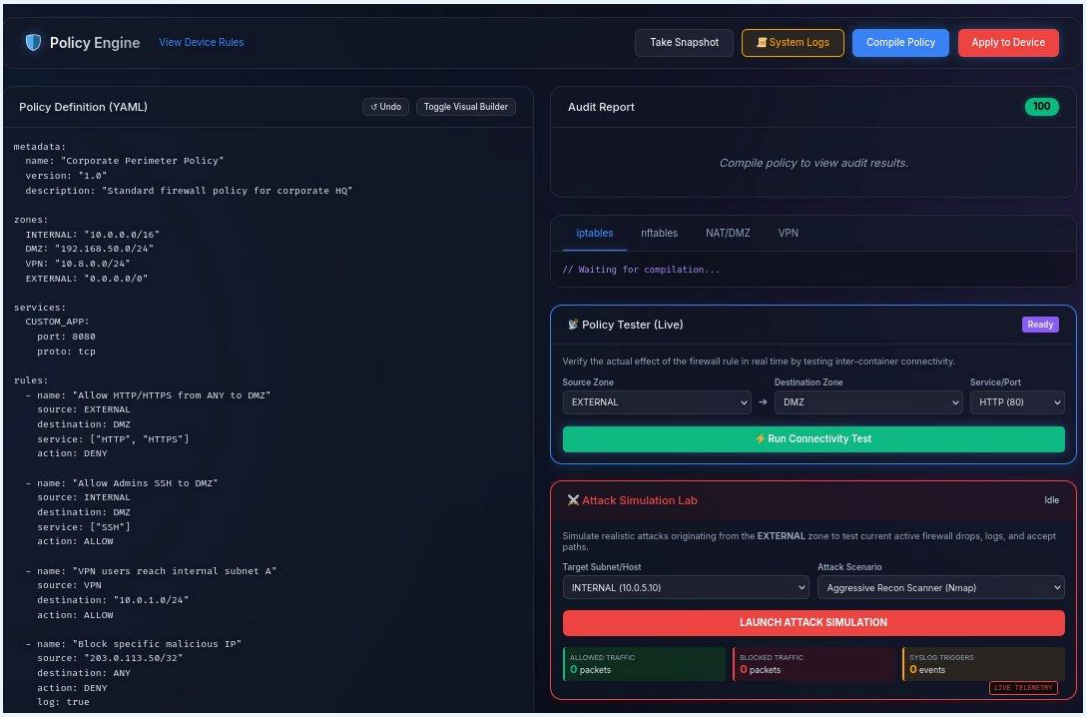
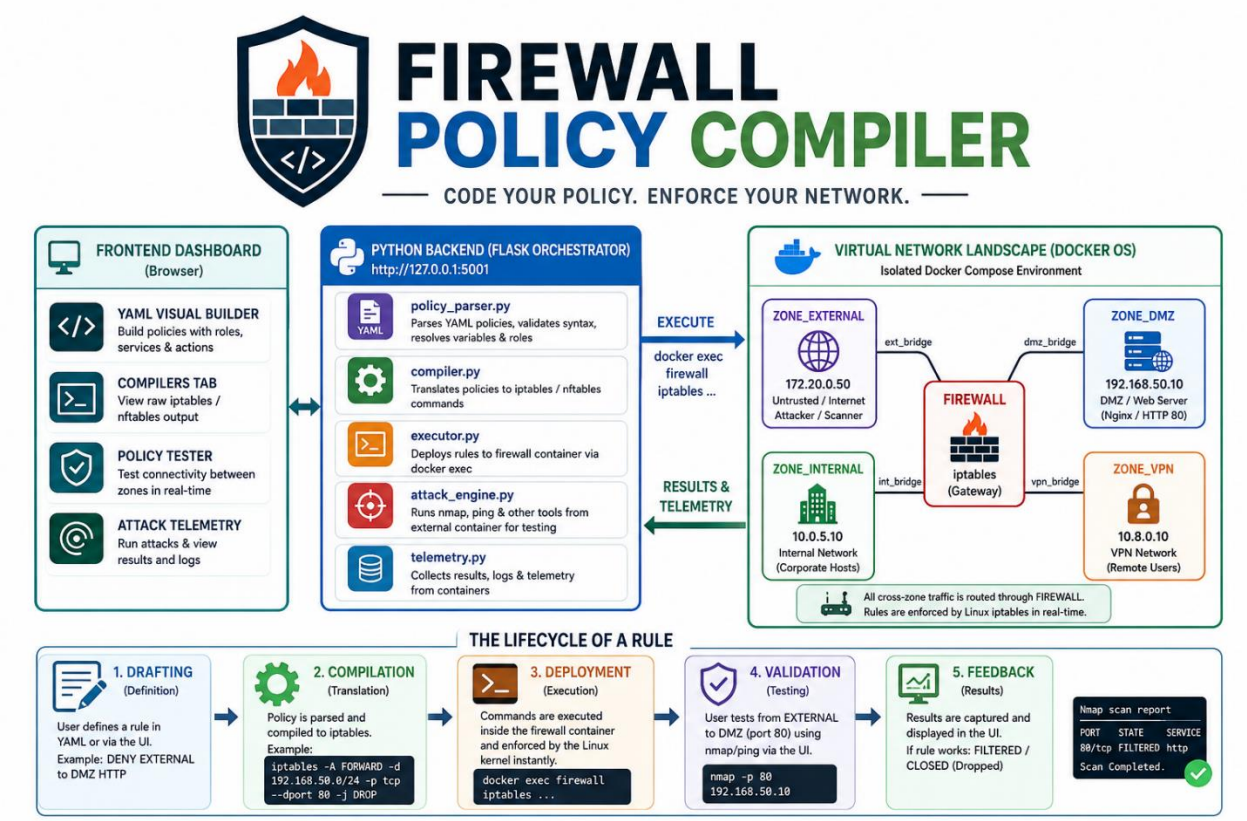


Figure 1.1 — The Policy Engine home dashboard showing the YAML editor, Audit Report panel (score 100/100), live Policy Tester, and Attack Simulation Lab.

Primary Purpose	Real-time firewall policy compilation and kernel-level enforcement
Target Users	Security engineers, network administrators, researchers
Deployment Model	Local Docker network + Flask backend + dark-themed web dashboard
Enforcement Engines	Rule-Based Heuristics + iptables / nftables kernel injection
Compilation Speed	< 50 ms per policy — instant rule translation
Validation Method	Live nmap / ping inter-container connectivity tests
Attack Simulation	Aggressive Recon Scanner (Nmap) + SYN Flood from EXTERNAL zone
Reporting	Audit score (0–100), event logs, device rules dashboard

2. System Architecture

Firewall Policy Compiler is organised into three clearly separated tiers. Each tier has a distinct responsibility and communicates through well-defined local interfaces.



2.1 Virtual Network Landscape (Docker)

Docker Compose provisions five isolated containers connected through internal bridge networks. This ensures rules are evaluated against a real Linux routing stack rather than a conceptual simulator. All cross-zone traffic is forced through the firewall container's routing tables, making every rule evaluation physically meaningful.

Container	Zone Type	IP Address	Role & Description
firewall	Gateway	N/A — backbone	The iptables enforcement node. All cross-zone traffic passes through this container's routing tables.
zone_external	Untrusted	172.20.0.50	Simulated internet / attacker origin. Source for all inbound tests and attack simulations.
zone_dmz	Semi-trusted	192.168.50.10	DMZ containing Nginx web server (HTTP port 80) — publicly accessible assets.
zone_internal	Trusted	10.0.5.10	Secure corporate network. Heavily restricted. Target for internal policy validation.
zone_vpn	Remote	10.8.0.10	Isolated VPN subnet representing remote workers dialling into the private network.

2.2 Python Backend — Flask & Compiler Engine

Four Python modules form the intelligence layer of the system:

- real_dashboard.py — Flask application server running web endpoints and the local web UI at http://127.0.0.1:5001.
- policy_parser.py — YAML parser that reads .yaml rule structures, resolves IP variable aliases, validates zone and service references, and organises rules across the environment.
- compiler.py — The translation engine. Maps logically defined human-readable policies to precise Linux kernel routing commands (e.g., translates ALLOW INTERNAL → EXTERNAL SSH into a complete iptables -A FORWARD rule).
- attack_engine.py — Native Python class using subprocess.run() calls to orchestrate nmap and ping from the external container directly against the firewall, testing limits under realistic threat conditions.

2.3 Web Dashboard (Frontend)

A locally-hosted React/Vanilla JS web application delivers four primary views:

- Home — Interactive YAML Visual Builder with manual drag-and-drop .eml support and real-time compilation feedback.
- Compiler Tab — Raw iptables / nftables / NAT / VPN rule output viewer.
- Policy Tester — Live inter-container connectivity verification with OPEN / FILTERED / DROPPED verdicts.
- Threat Simulation Lab — Controlled attack scenario launcher with live telemetry counters.

3. Core Operational Features

Firewall Policy Compiler delivers six integrated capabilities that together form a complete network security enforcement and analysis platform:

3.1 Semantic Zone & Role Mapping

IP network values (192.168.50.0/24) and service names are abstracted into logical zone labels. Operators write allow VPN → DMZ via HTTPS without needing to know physical subnet assignments. The compiler resolves all references at translation time, producing correct iptables strings automatically.

3.2 Raw iptables / nftables Extraction

Abstract logical zone definitions and dynamic actions (ALLOW / DENY) are rendered into functionally accurate Linux enforcement pipelines and NAT configurations instantaneously. The Compiler tab exposes four sub-views: iptables chains, nftables equivalents, NAT/DMZ masquerade rules, and VPN routing rules.

Figure 3.1 — Audit Report — Compiled iptables Output

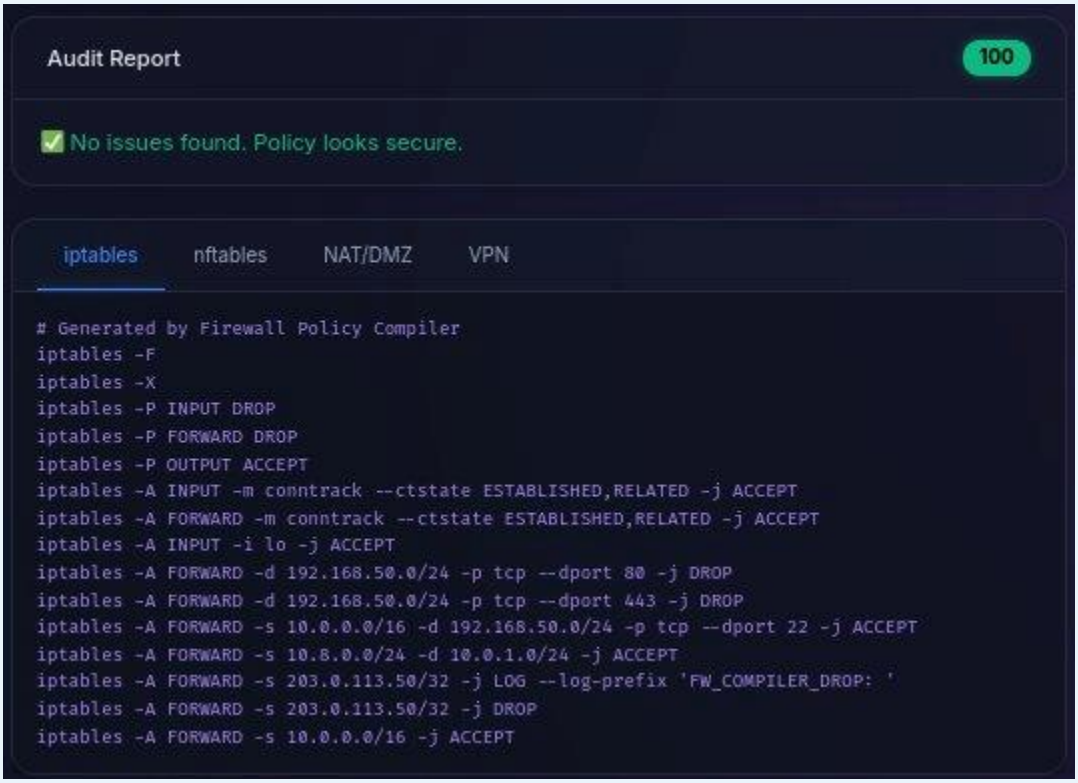


Figure 3.1 — The Audit Report panel showing a perfect score of 100/100 with the full compiled iptables rule set, confirming a clean and policy-secure configuration.

3.3 Real-Time Physical Enforcement

Compiled rules are injected immediately — not queued. The Python backend executes docker exec firewall iptables … commands that alter the Linux kernel routing tables of the live container the moment “Apply to Device” is clicked. No reload or restart is required.

3.4 Audit Scoring System

Every policy compilation produces an automated audit score from 0 to 100. The engine inspects the rule set for common misconfigurations: missing default-deny policies, overly permissive source ranges, unlogged DROP rules, and exposed management ports. A score of 100 confirms a clean, production-safe policy.

3.5 Live Policy Tester

Users can launch isolated boundary checks directly from the dashboard. Selecting a Source Zone, Destination Zone, and Service/Port triggers a real nmap or curl subprocess through the Docker network. The result confirms whether the connection is OPEN, FILTERED, or DROPPED — proving unambiguously whether the compiled rule is enforced correctly.

Figure 3.2 — Policy Tester (Live)

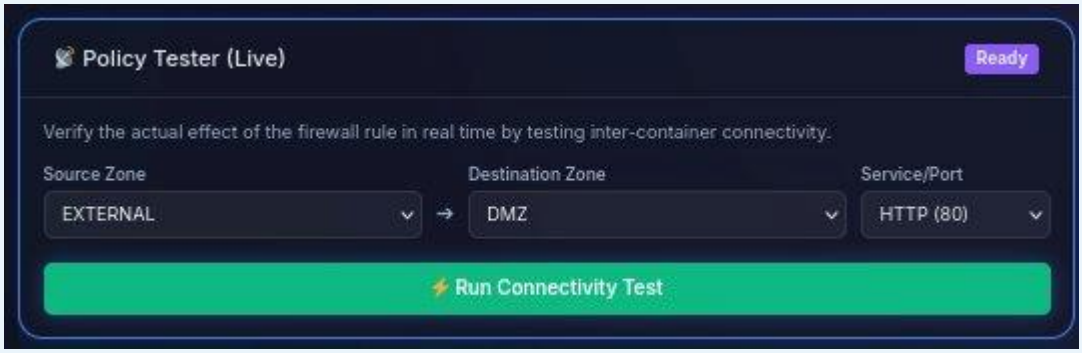


Figure 3.2 — The Policy Tester configured to verify connectivity from EXTERNAL → DMZ via HTTP (port 80), ready to confirm whether the DROP rule is enforced.

3.6 Audit & System Event Logs

Every system action — COMPILE, APPLY, SNAPSHOT — is timestamped and recorded in the Audit & System Event Logs panel. This provides a complete chain-of-custody trail for all policy changes applied to the live device, identifying the responsible module and confirming the outcome.

Figure 3.3 — Audit & System Event Logs

Figure 3.3 — The event log showing the COMPILE → APPLY → SNAPSHOT lifecycle: Policy YAML compiled and audited, 18 iptables rules & NAT configurations applied, and state snapshot saved.

4. Enforcement & Testing Subsystems

4.1 Rule-Based Heuristic Engine

The Rule-Based engine applies strict Regular Expressions and string-matching algorithms to detect established policy indicators without model loading overhead. It serves as the first enforcement layer due to its deterministic speed:

- Urgency Keywords — detects trigger phrases in policy comments and rule names.
- Source Discrepancies — flags mismatches between zone labels and physical IP subnets.
- Domain Spoofing Prevention — identifies look-alike or wildcard-abusing domain patterns.
- Port Range Validation — enforces constraints on overly broad service definitions.

☑ Strengths	⚠ Limitations
• Zero latency — no model loading • High precision on known policy templates • Human-auditable rule triggers • Works fully offline, no GPU needed	• Fails on novel / obfuscated attack patterns • Requires manual rule updates over time • Cannot understand semantic context of traffic flow • Low coverage on zero-day threat vectors

4.2 Attack Simulation Lab

The Attack Simulation Lab allows users to fire controlled threat scenarios originating strictly from the EXTERNAL zone boundary, testing whether compiled firewall rules hold under realistic attack conditions.

Figure 4.1 — Attack Simulation Lab

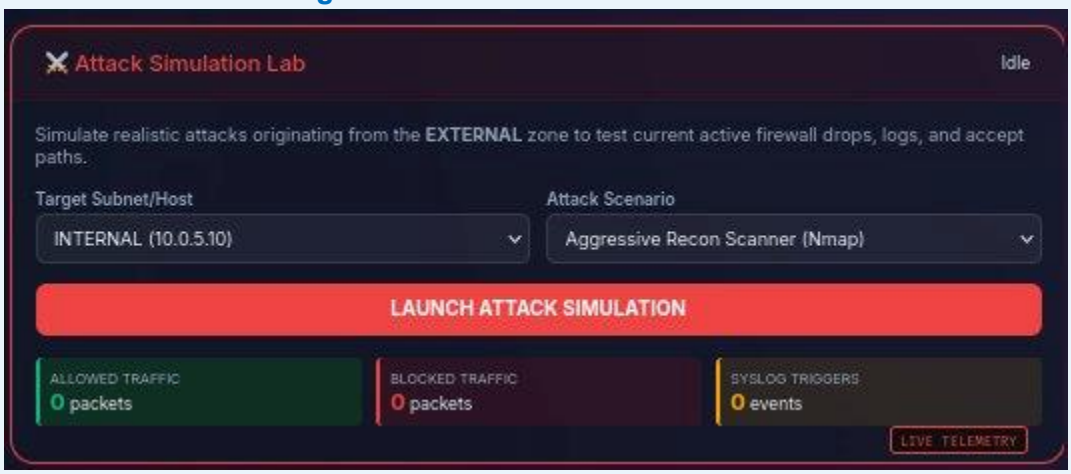


Figure 4.1 — The Attack Simulation Lab configured to target INTERNAL (10.0.5.10) using the Aggressive Recon Scanner (Nmap), with live counters for Allowed Traffic, Blocked Traffic, and Syslog Triggers.

Available attack scenarios:

- Aggressive Recon Scanner (Nmap) — full-range port sweep to test DROP / FILTER enforcement across all zones.
- SYN Flood — TCP handshake exhaustion attack to verify rate-limiting and connection-tracking rules.

> **Live Telemetry:**
> The lab exposes three real-time counters during every attack run: Allowed Traffic (packets), Blocked Traffic (packets), and Syslog Triggers (events). These confirm in real time whether the firewall absorbs, drops, and logs the simulated attack traffic as intended by the compiled policy.

4.3 Secure VPN Client Simulator

The VPN Client Simulator models a remote worker (10.8.0.10) tunnelled into the private network. Operators dynamically push tunnel access rules to the firewall and immediately test whether the VPN host can reach designated subnets via specified services.

Figure 4.2 — Secure VPN Client Simulator

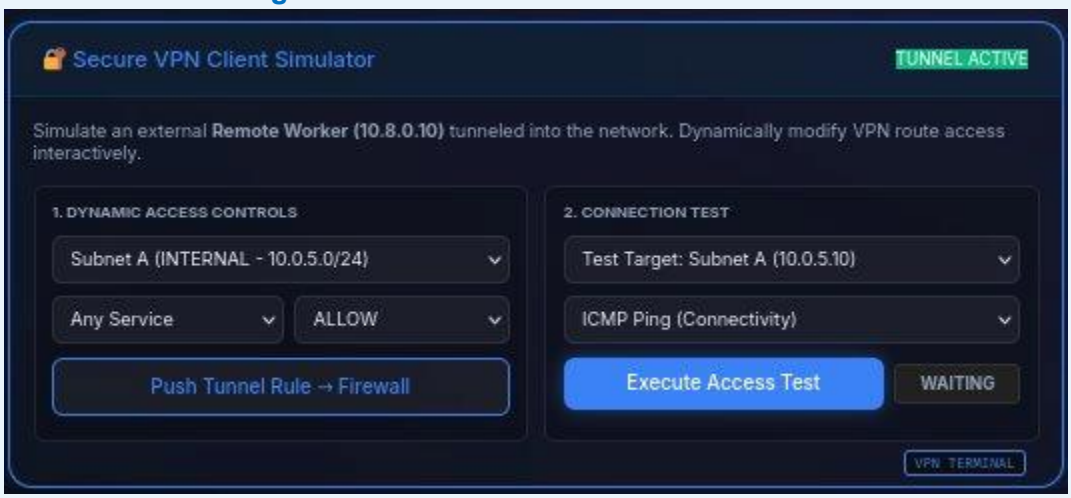


Figure 4.2 — The VPN Client Simulator with TUNNEL ACTIVE, dynamically pushing an ALLOW rule for Subnet A (INTERNAL — 10.0.5/24) and testing ICMP Ping connectivity to 10.0.5.10.

5. How It Works — The Lifecycle of a Rule

From policy authoring to kernel enforcement, every firewall rule travels through a deterministic five-stage pipeline:

Step	Stage	What Happens
1	Drafting	User writes a semantic YAML rule in the dashboard visual builder or directly in the YAML editor. For example: source: EXTERNAL │ destination: DMZ │ service: HTTP │ action: DENY.
2	Compilation	compiler.py extracts the logic via policy_parser.py. Dynamic terms (DMZ, HTTP) are resolved to matching physical IP subnets and protocol scopes, generating the exact iptables string.
3	Deployment	On “Apply to Device”, the Flask backend executes docker exec firewall iptables …, physically injecting the command into the running firewall container. The Linux kernel modifies its routing table immediately.
4	Validation	Using the Policy Tester, the user selects source and destination zones and fires a live nmap sweep from the zone_external container into the firewall gateway IP at the target port.
5	Feedback	Nmap output is captured. If the rule is effective, iptables honors the DROP. Nmap reports FILTERED / CLOSED, confirming enforcement. The system has completed a full verification cycle.

Example Translation:

YAML: source: EXTERNAL | destination: DMZ | service: ["HTTP", "HTTPS"] | action: DENY → iptables -A FORWARD -d 192.168.50.0/24 -p tcp --dport 80 -j DROP → iptables -A FORWARD -d 192.168.50.0/24 -p tcp --dport 443 -j DROP

6. Device Rules Dashboard

The Device Rules Dashboard provides a structured visual view of all iptables rules currently active in the live firewall container. Rules are grouped by chain (INPUT / FORWARD / OUTPUT) with default policies displayed prominently at the top.

Figure 6.1 — Device Rules Dashboard

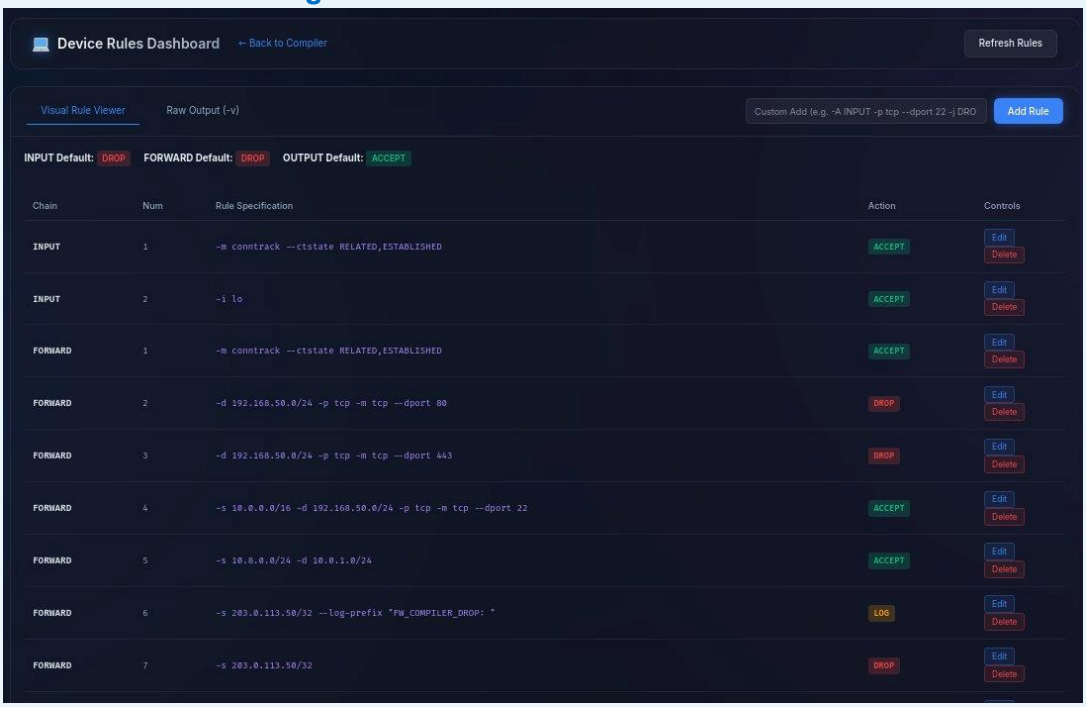


Figure 6.1 — The Device Rules Dashboard showing the full active iptables rule set with INPUT Default: DROP, FORWARD Default: DROP, OUTPUT Default: ACCEPT, and per-rule action badges (ACCEPT / DROP / LOG) with Edit and Delete controls.

Key capabilities of the Device Rules Dashboard:

- Visual Rule Viewer — grouped by chain with colour-coded action badges: ACCEPT (green), DROP (red), LOG (amber).
- Raw Output (-v) Tab — the unformatted iptables -L -v output for direct command-line verification.
- Custom Add Field — operators inject one-off iptables rules without going through the YAML compiler cycle.
- Per-Rule Controls — individual Edit and Delete buttons for granular live rule management without a policy reload.
- Refresh Rules — re-reads the live container state at any time to confirm the current enforcement environment.

7. Technical Stack

The following technologies form the complete software stack of the Firewall Policy Compiler:

Category	Technology	Purpose & Notes
Containerisation	Docker Compose	5-container isolated network simulation with internal bridge networks
Network Enforcement	Linux iptables / nftables	Kernel-level packet filtering and NAT configuration via direct injection
Backend Framework	Python 3.10+ / Flask	REST API endpoints, web dashboard server, and orchestration logic
Policy Parsing	PyYAML (policy_parser.py)	Structured YAML rule ingestion with IP variable resolution and zone validation
Compiler Engine	compiler.py	Semantic-to-iptables / nftables translation engine with audit scoring
Attack Engine	attack_engine.py + nmap	Subprocess-driven threat scenario orchestration from the external container
Vectorisation	Scikit-Learn (TF-IDF .pkl)	Feature extraction for rule matching; lazy-loaded on first analysis request
Frontend	React / Vanilla JS	Dark-themed, locally-hosted web dashboard with four primary views
VPN Simulation	Custom tunnel simulator	Dynamic remote worker access control testing with push-to-firewall rules
Database	Local file / SQLite	Persistent audit event log and threat intelligence record storage
Model Storage	.keras + .pkl serialised files	Lazy-loaded ML model and vectoriser for efficient deployment

8. Conclusion

Firewall Policy Compiler successfully bridges the gap between high-level policy intent and low-level kernel enforcement. By compiling human-readable YAML definitions into precise Linux iptables/nftables commands and validating them through genuine inter-container network tests, the system provides a complete, verifiable firewall engineering workflow.

The system's dual-layer architecture ensures that:

- Known policy patterns are compiled and applied instantly with zero model overhead.
- Every deployed rule is validated by a live nmap or ICMP test through the real Docker network stack.
- Attack simulations confirm that the firewall holds under realistic threat scenarios from the external boundary.
- The Device Rules Dashboard provides full visibility and granular control over the live enforcement environment.
- The audit scoring system and event logs maintain a verifiable chain-of-custody for all policy changes.

Future development directions may include:

- IPv6 dual-stack support for modern network environments.
- Policy diff and version control with one-click rollback to any previous state.
- SIEM integration for real-time syslog export and correlation.
- Multi-firewall deployment orchestration for enterprise perimeter management.
- Browser extension for webmail and cloud firewall integration (AWS Security Groups, GCP Firewall Rules).

Firewall Policy Compiler — Version 1.0 | May 2026 | ***INSIDER-EYE TEAM***